EXPERIMENT 1

Aayush Patil
D20A-56

AIM: Cryptography in Blockchain, Merkle root Tree Hash

THEORY:

1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function takes any input data and converts it into a fixed length string of characters called a hash. In blockchain, SHA-256 is the most commonly used hashing algorithm. It is practically important because of a few key properties. The same input always gives the same output, so the hash is deterministic. Even a tiny change in input, like changing one letter, completely changes the output hash, this is called the avalanche effect. It is a one way function, meaning you cannot get back the original input from the hash. It is also collision resistant, so it is extremely hard to find two different inputs that give the same hash. These properties make hashing the backbone of blockchain security, since every block, transaction and wallet address relies on hashing to stay tamper proof.

2. What is a Merkle Tree?

A Merkle Tree, also called a hash tree, is a data structure used to efficiently and securely verify the contents of large sets of data, like all the transactions inside a block. Instead of storing every transaction separately for verification, the Merkle Tree combines and hashes them together in pairs, layer by layer, until only one hash remains at the top. This final hash is called the Merkle Root, and it represents a summary of all the transactions in that block.

3. Cryptographic Puzzle and the Golden Nonce

In blockchain mining, a cryptographic puzzle is a computational problem miners must solve to add a new block to the chain. This process is called proof of work. Miners take the block data and combine it with a random number called a nonce, then hash the result. The goal is to find a nonce that produces a hash with a certain number of leading zeros, based on the network's difficulty level. Since hash functions are unpredictable, miners have to try nonce values one by one through brute force until they find the correct one. This winning value is called the golden nonce, and the miner who finds it first gets to add the block and earn the reward. This process makes mining computationally expensive and secures the network against tampering.

4. How Does a Merkle Tree Work?

The construction of a Merkle Tree starts from the bottom layer, called leaf nodes, where each transaction is individually hashed. These leaf hashes are then grouped in pairs and hashed together to form parent nodes. If there is an odd number of hashes at any level, the last one is duplicated so it can be paired. This pairing and hashing process continues layer by layer,

moving
upward, until only a single hash remains, which is the Merkle Root. This root hash is stored in the block header and acts as a fingerprint for every transaction included in that block.

5. Benefits of Merkle Tree

Merkle Trees offer efficient and fast verification of data since you don't need to check every transaction individually. They allow lightweight verification, meaning even devices with limited storage, like mobile wallets, can confirm a transaction is part of a block without downloading the entire blockchain. They also help detect tampering easily, since changing even one transaction changes the root hash completely. On top of this, they save storage and bandwidth because only the root hash needs to be stored and compared, not the entire dataset.

6. Use Cases of Merkle Tree

Merkle Trees are used in blockchain networks like Bitcoin and Ethereum to verify transactions within blocks. They are also used in peer to peer file sharing systems like BitTorrent to check if downloaded file pieces are correct. Git, the version control system, uses a similar structure to track changes in files and commits. Merkle Trees are also applied in distributed databases and cloud storage systems to make sure data hasn't been corrupted or altered across multiple nodes.

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus,emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management

CODE:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib
```

```
def create_hash(string):
 # Create a hash object using SHA-256 algorithm
 hash_object = hashlib.sha256()
 # Convert the string to bytes and update the hash object
hash_object.update(string.encode('utf-8'))
 # Get the hexadecimal representation of the hash
 hash_string = hash_object.hexdigest()
 # Return the hash string
 return hash_string
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Program #2: Program to generate required target hash with input string and

nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce
# Calculate the hash using SHA-256
hash_object =
hashlib.sha256(hash_string.encode('utf-8')) hash_code =
hash_object.hexdigest()
# Print the hash code
print("Hash Code:", hash_code)
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
 nonce = 0
 hash_prefix = '0' * num_zeros
```

```
    while True:
    # Concatenate the string and nonce
    hash_string = input_string + str(nonce)
    # Calculate the hash using SHA-256
    hash_object = hashlib.sha256(hash_string.encode('utf-8'))
   hash_code = hash_object.hexdigest()
    # Check if the hash has the required leading zeros  if
   hash_code.startswith(hash_prefix):
    print("Hash:", hash_code)
    return nonce
    nonce += 1


   # Get user input
   input_string = "shravanya"
   num_zeros = 8
 # Find the expected nonce
   expected_nonce = find_nonce(input_string, num_zeros)
   # Print the expected nonce
   print("Input String:", input_string)
   print("Leading Zeros:", num_zeros)
   print("Expected Nonce:", expected_nonce)
```

Program #4: Generating Merkle Tree for given set of

Transactions `import hashlib`

```
def build_merkle_tree(transactions):
 if len(transactions) == 0:
 return None
 # Hash all transactions first
 hashed_transactions = []
 print("===== TRANSACTION HASHES =====")
 for transaction in transactions:
 hash_transaction = hashlib.sha256(
 transaction.encode('utf-8')
 ).hexdigest()
 hashed_transactions.append(hash_transaction)
```

```
print("Transaction:", transaction)
 print("Hash:", hash_transaction)
 print()
 transactions = hashed_transactions
 # Store all levels of the Merkle Tree
 tree = [transactions.copy()]

 level = 1

 # Construct the Merkle Tree
 while len(transactions) > 1:

 # If number of hashes is odd, duplicate the last hash  if
len(transactions) % 2 != 0:
 transactions.append(transactions[-1])
new_transactions = []
 print("===== MERKLE TREE LEVEL", level, "=====")  for i in
range(0, len(transactions), 2):  combined = transactions[i]
+ transactions[i + 1]  hash_combined = hashlib.sha256(
 combined.encode('utf-8')
 ).hexdigest()
 new_transactions.append(hash_combined)
print("Hash 1:", transactions[i])
 print("Hash 2:", transactions[i + 1])
print("Combined Hash:", hash_combined)  print()
 transactions = new_transactions
 tree.append(transactions.copy())
 level += 1
 return transactions[0], tree
# Transactions
transactions = [
 "Alice -> Bob : $200",
 "Bob -> Dave : $500",
 "Dave -> Eve : $100",
 "Eve -> Alice : $300",
 "Roo -> Bob : $50"
]
# Build Merkle Tree
merkle_root, merkle_tree =
build_merkle_tree(transactions) # Print all levels of the
Merkle Tree
```

```
print("========== MERKLE TREE ==========")
for i, level in enumerate(merkle_tree):
 print("Level", i, ":")
 for j, hash_value in enumerate(level):
 print(" H{} = {}".format(j + 1, hash_value))

 print()
# Print Merkle Root
print("================================")
print("Merkle Root:", merkle_root)
```

OUTPUT:

1)

```
Enter a string: shravanya
Hash: db8d368edf99445b3f5137ee5f70f7b3cd94e184ac610c4cc10856aeec0b1883
```

2)

```
Enter a string: shravanya
Enter the nonce: 17
Hash Code:
e65241a89af43eaa5a002eac697059da7234d9d6a79841dc8bfea100cea21ab7
```

3)

```
Hash:
000000ddf230d9dcd1cb472e10192584f09f2c8ce49497fd70b938ea90b8d815
Input String: shravanya
Leading Zeros: 6
Expected Nonce: 6184114
```

4)

```
===== TRANSACTION HASHES =====
Transaction: Alice -> Bob : $200
Hash: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Transaction: Bob -> Dave : $500
Hash: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

Transaction: Dave -> Eve : $100
Hash: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

Transaction: Eve -> Alice : $300
Hash: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

Transaction: Roo -> Bob : $50
```

```
Hash: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

===== MERKLE TREE LEVEL 1 =====
Hash 1:
f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc71327976
0 Hash 2:
768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b4
6 Combined Hash:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Hash 1:
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890d
a Hash 2:
cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8
a Combined Hash:
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Hash 1:
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58a
c Hash 2:
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58a
c Combined Hash:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

===== MERKLE TREE LEVEL 2 =====
Hash 1:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e
9 Hash 2:
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de
7 Combined Hash:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Hash 1:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18
e Hash 2:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Combined Hash:
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

===== MERKLE TREE LEVEL 3 =====
Hash 1:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8e
e Hash 2:
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f4
5 Combined Hash:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

========== MERKLE TREE ==========
Level 0 :
 H1 = f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
H2  =  768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
H3  =  43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
H4  =  cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
H5 = cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
```

```
Level 1 :
 H1 = 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
H2  =  a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
H3 = 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Level 2 :
 H1 = ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
H2 = 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Level 3 :
  H1 = 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

================================
Merkle Root:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

CONCLUSION :

Cryptography in Blockchain, Merkle root Tree Hash has been successfully implemented.